

LẬP TRÌNH JAVA



Giảng viên: ThS Huỳnh Vĩnh Phát

Biên soạn: TS Trần Duy Thanh

Bài Học

Khái niệm về mạng



Nội dung bài học

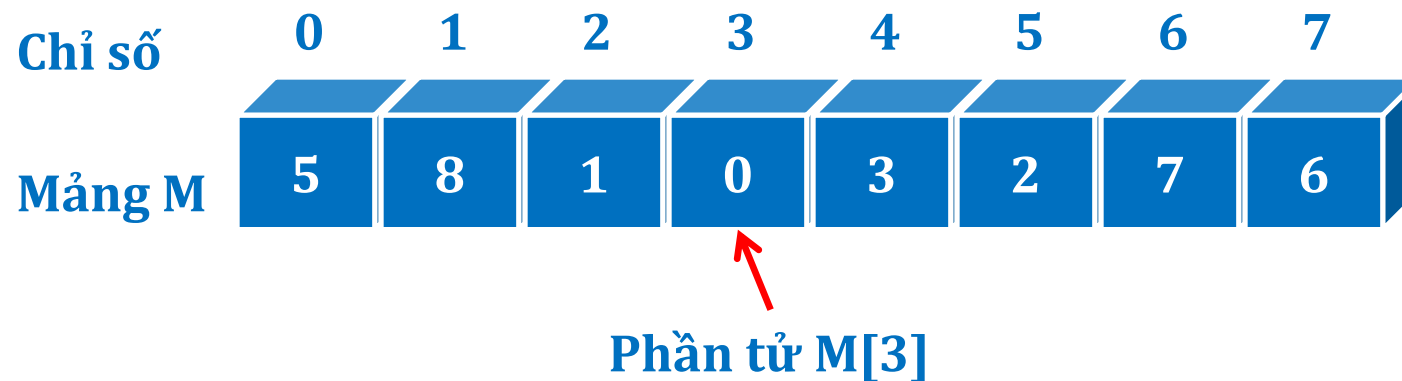
- ❖ Khái niệm về mảng
- ❖ Mục đích dùng mảng

Khái niệm về mảng

Mảng là một **loại biến đặc biệt**, bao gồm một **dãy các ô nhớ có nhiều ô nhớ con** cho phép biểu diễn thông tin dạng danh sách trong thực tế

Các phần tử trong mảng có **cùng kiểu dữ liệu với nhau**

Ví dụ:



Một số mảng khác: Số thực, mảng chuỗi

Mục đích dùng mảng

- ✓ Mảng là cách tốt nhất cho phép quản lý nhiều phần tử dữ liệu có cùng kiểu tại cùng một thời điểm
- ✓ Mảng tạo ra sự tối ưu trong việc quản lý bộ nhớ so với việc sử dụng nhiều biến cho cùng một chức năng

Bộ nhớ có thể được gán cho mảng chỉ khi mảng thực sự được sử dụng. Do đó, bộ nhớ không bị tiêu tốn cho mảng ngay khi bạn khai báo mảng.

Bài Học

Cách khai báo và cấp phát bộ nhớ cho mảng



Nội dung bài học

- ❖ Khai báo mảng như thế nào
- ❖ Cấp phát bộ nhớ sử dụng mảng

Khai báo mảng như thế nào

Khai báo không khởi tạo kích thước và giá trị

KiểuDữLiệu[] TênMảng;

KiểuDữLiệu TênMảng[];

Ví dụ:

int[] M;

Hoặc int M[];

Cấp phát bộ nhớ sử dụng mảng

- Khai báo khởi tạo kích thước nhưng không có giá trị ban đầu

KiểuDữLiệu[] TênMảng = new KiểuDữLiệu[SốPhầnTử];

KiểuDữLiệu TênMảng[] = new KiểuDữLiệu[SốPhầnTử];

- Ví dụ:

```
int[] a = new int[5];
```

```
int a[] = new int[5];
```

Cấp phát bộ nhớ sử dụng mảng

- Khai báo có khởi tạo kích thước và khởi tạo giá trị ban đầu:

KiểuDữLiệu[] TênMảng = new KiểuDữLiệu[]

{giá trị 1, giá trị 2, giá trị 3, ...};

hoặc

KiểuDữLiệu[] TênMảng =

{giá trị 1, giá trị 2, giá trị 3, ...};

- Ví dụ:

`int[] a = new int[]{2,10,4,8,5};`

`int[] a = {2, 10, 4, 8, 5};`

Bài Học

Truy suất và thao tác trên mảng



Nội dung bài học

- ❖ Cách lấy dữ liệu từ mảng
- ❖ Cách thay đổi dữ liệu trên mảng
- ❖ Cách xuất dữ liệu mảng: for, for giá trị

⬢ Cách lấy dữ liệu từ mảng

❑ Thao tác cơ bản

- Truy xuất giá trị 1 phần tử:

TênMảng[vị _trí_i]

- Vị trí của 1 phần tử trong mảng bắt đầu từ 0
- Vị_trí_i có giá trị từ 0 đến (số phần tử - 1)

Chỉ số	0	1	2	3	4	5	6	7
Mảng M	5	8	1	113	3	2	7	6

M[3] ?
M.length ?

- Lấy chiều dài của mảng: thuộc tính **length**

TênMảng.length

Cách thay đổi dữ liệu trên mảng

❖ $M[i] = \text{value}$

❖ $M[3] = 113$



Cách xuất dữ liệu mảng: for, for giá trị

- Duyệt và xử lý từng phần tử của mảng:

```
for (int i = 0; i < TênMảng.length; i++)  
{  
    // Xử lý trên phần tử TênMảng[i]  
}
```

- Ví dụ: duyệt và xuất mảng

```
int[] M = {1,2,3,4,5};  
for (int i = 0; i < M.length; i++)  
    System.out.println(M[i]);
```

Cách xuất dữ liệu mảng: for, for giá trị

- Duyệt và xử lý từng phần tử của mảng:

```
for (int i: TênMảng)
{
    // Xử lý trên phần tử i
}
```

- Ví dụ: duyệt và xuất mảng

```
int[] M = {1,2,3,4,5};
for (int i: M)
    System.out.println(i);
```


Bài Học

Tìm kiếm trên mạng



Nội dung bài học

- ❖ Tìm 1 phần tử có tồn tại hay không
- ❖ Tìm các vị trí của phần tử trong mảng

Nội dung bài học

- ❖ Tìm 1 phần tử có tồn tại hay không
- ❖ ➔ tương tự như duyệt mảng, trong quá trình duyệt ta so sánh, nếu tìm thấy thì dừng

Nội dung bài học

- ❖ Tìm các vị trí của phần tử trong mảng
- ❖ ➔ tương tự như duyệt mảng, trong quá trình duyệt ta so sánh, nếu tìm thấy thì lưu vết lại các vị trí rồi tiến hành tìm tiếp

Bài Học

Sắp xếp mảng



Nội dung bài học

- ❖ Sắp xếp mảng dùng các giải thuật sắp xếp: Bubble Sort, SelectionSort, QuickSort...
- ❖ Sắp xếp mảng dựa vào các thư viện có sẵn trong Java

Sắp xếp mảng dùng các giải thuật sắp xếp

```
void BubbleSort(int []M)
{
    int i, j;
    for (i = 0; i < M.length - 1; i++)
    {
        for (j = M.length - 1; j > i; j--)
        {
            if (M[j] < M[j - 1])// nếu có nghịch thế
            {
                int temp = M[j];
                M[j] = M[j - 1];
                M[j - 1] = temp;
            }
        }
    }
}
```

Sắp xếp mảng dùng các giải thuật sắp xếp

```
void SelectionSort(int []M)
{
    int min;
    for(int i=0;i<M.length-1;i++)
    {
        min = i;
        for(int j=i+1;j<M.length;j++)
        {
            if (M[j] < M[min])
                min = j;
        }
        if(min!=i)
        {
            int temp = M[i];
            M[i] = M[min];
            M[min] = temp;
        }
    }
}
```


Sắp xếp mảng dùng các giải thuật sắp xếp

```
void QuickSort(int []M,int left, int right)
{
    if (left >= right) return;
    int pivot = M[(left + right) / 2];
    int i = left, j = right ;
    do{
        while (M[i] < pivot) i++;
        while (M[j] > pivot) j--;
        if (i <= j){
            int temp = M[i];
            M[i] = M[j];
            M[j] = temp;
            i++;
            j--;
        }
    } while (i < j);
    QuickSort(M,left, j);
    QuickSort(M,i,right);
}
```

`int[] M = {100,3,60,35,2};`
`QuickSort(M,0,M.length-1);`



Sắp xếp mảng dựa vào các thư viện có sẵn

```
int M[] = { 2, 5, -2, 6, -3, 8, 0, 7, -9, 4 };
```

```
// Sắp xếp mảng số nguyên
```

```
Arrays.sort(M);
```

Giới thiệu youtube giải thuật sắp xếp

https://www.youtube.com/playlist?list=PLmEUE4MG8_b63bmg14n_V1r1LkfyphJ6l

Bài Học

Các hạn chế của mảng



Nội dung bài học

- Mảng có kích cỡ và số chiều cố định nên khó khăn trong việc mở rộng ứng dụng.
- Các phần tử được đặt và tham chiếu một cách liên tiếp nhau trong bộ nhớ nên khó khăn cho việc xóa một phần tử ra khỏi mảng.
- Giải thích trường hợp xóa, chèn giữa, thêm vượt quá kích thước mảng

Bài Học

Lý do sử dụng collection



Nội dung bài học

- ❖ Collection là gì?
- ❖ Cách khắc phục hạn chế của Mảng

Collection là gì?

- ❖ Collection bản chất là tập các lớp dùng để lưu trữ danh sách và có khả năng tự co giãn khi danh sách thay đổi: Thêm , sửa, xóa, chèn...
- ❖ Hai lớp Collection thường được sử dụng nhiều nhất là **ArrayList** và **HashMap**

Bài Học

Cách sử dụng ArrayList



Cách sử dụng ArrayList

□ Giới thiệu về ArrayList

- ArrayList sử dụng cấu trúc mảng để lưu trữ phần tử, tuy nhiên có hai đặc điểm khác mảng:
 - Không cần khai báo trước kiểu phần tử.
 - Không cần xác định trước số lượng phần tử (kích thước mảng).
 - Nó có khả năng truy cập phần tử ngẫu nhiên (Do thừa kế từ interface RandomAccess).

Cách sử dụng ArrayList

❑ Phương thức khởi tạo

- ArrayList()
- ArrayList(Collection c)
- ArrayList(int initialCapacity)

Cách sử dụng ArrayList

❑ Các phương thức chính

- `add(Object o)`
- `remove(Object o)`
- `get(int index)`
- `size()`
- `isEmpty()`
- `contains(Object o)`
- `clear()`

Cách sử dụng ArrayList

```
ArrayList<String>ds=new ArrayList<String>();  
ds.add("Hạnh");  
ds.add("Phúc");  
ds.add("Giải");  
ds.add("Thoát");  
ds.add("Vô");  
ds.add("Thuờng");  
for(String s : ds)  
{  
    System.out.println(s);  
}  
    for(int i=0;i<ds.size();i++)  
    {  
        String s=ds.get(i);  
    }
```

Bài Học

Cách sử dụng HashMap



HashMap

□ Giới thiệu về HashMap

- Là kiểu tập hợp từ điển, HashMap cho phép truy xuất trực tiếp tới một đối tượng bằng khoá duy nhất của nó. Cả hai khoá và đối tượng có thể thuộc bất cứ kiểu nào.

HashMap

□ phương thức khởi tạo

- HashMap()
- HashMap(Collection c)
- HashMap(int capacity)

HashMap

□ Các phương thức chính

- put(Object key, Object value)
- get(Object key)
- remove(Object key)
- containsKey(Object key)
- containsValue(Object value)
- size()
- isEmpty

HashMap

```
HashMap<Integer, String>map=
    new HashMap<Integer, String>();
map.put(1, "Trần Duy Thanh");
map.put(2, "Nguyễn Văn Hùng");
map.put(3, "Đỗ Công Thành");
map.put(4, "Nguyễn Cẩm Hương");
map.put(5, "Phạm Thị Xuân Diệu");

String ten=map.get(1);
if (map.containsKey(6)==false)
    map.put(6, "Obama");

Collection<String>dsTen= map.values();
for (String x : dsTen)
    System.out.println(x);
```

Bài Học

Bài tập rèn luyện về mảng



Nội dung bài học

- ❖ Nhập vào một mảng ngẫu nhiên N số, sau đó:
 - Xuất toàn bộ mảng
 - Tính tổng mảng
 - Với K là 1 số nhập từ bàn phím, hỏi k xuất hiện bao nhiêu lần
 - Tìm phần tử lớn nhất
 - Tìm phần tử nhỏ nhất
 - Xuất các số nguyên tố
 - Sắp xếp mảng tăng dần
 - Sắp xếp mảng giảm dần

Bài Học

Bài tập rèn luyện về ArrayList



Nội dung bài học

- ❖ Viết chương trình nhập danh sách các số nguyên lưu vào ArrayList, sau đó:
 - Thêm
 - Sửa
 - Xóa
 - Tìm kiếm
 - Sắp xếp

Bài Học

Bài tập rèn luyện về HashMap



Nội dung bài học

- ❖ Sử dụng HashMap <Integer,String> để lưu danh sách các cuốn sách cùng với mã của nó, sau đó:
 - Thêm
 - Xuất danh sách
 - Sửa
 - Xóa
 - Tìm kiếm

END

